

The Machine, Part 2: Machine shi Bolna

August 27, 2026

THE MACHINE

PART 2 OF 5: MACHINE शी बोलणं

पाच PARTS चा नकाशा

PART 1 पैसा आणि MACHINE [तुम्ही इथे आहात]	झाला
PART 2 MACHINE शी बोलणं	languages, operating system, software बनतं कसं
PART 3 MACHINES चं जाळं	network, address, internet
PART 4 DATA आणि आठवणी	storage, database, शोधणं
PART 5 जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 1 ची परीक्षा

पाच प्रश्न, Part 1 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा. जिथे अडाल, तिथला chapter number सोबत आहे; तिथे परत जा.

1. पैसा फक्त Level 1 वर का घुसतो? (0.3)
2. कमाईच्या सूत्रात तास का नाहीत? (0.2)
3. "Computer विचार करतो" म्हणणं चुकीचं का आहे? (1.3, 1.5)
4. एकच machine camera, bank आणि game कशी बनते? (1.6)
5. Text पेक्षा video लाखपट जड का आहे? (2.1, 2.2)

उत्तर: 1. कारण गरज फक्त माणसाची असते; Level 2, 3, 4 आपसात जो पैसा देतात, तो सगळा खालून चढत आलेला असतो. 2. कमाई = size x scarcity x leverage; तास विकणं म्हणजे leverage चा सगळ्यात खालचा काटा. 3. Machine आधीच design केलेली gates ची साखळी चालवते; प्रत्येक input चं output आधीच ठरलेलं; surprise फक्त आपल्यासाठी असतं. 4. कारण recipe पण number आहे; नवी recipe घाला, machine तीच, काम नवं. 5. अक्षर 1 byte; photo त millions ठिपके x 3 numbers; video त प्रति second 30 photos. मोजणी स्वतःच गुणाकार होत जाते.

या part मध्ये काय आहे

Part 1 ने दाखवलं की machine numbers वर recipe चालवते. आता दोन प्रश्न उघडायचे आहेत:

Section 3: Recipe लिहिली कशी जाते? माणसाच्या शब्दांपासून machine च्या numbers पर्यंत भाषांतर कोण करतं? Languages इतक्या सान्या का? आणि तो manager कोण जो हजार recipes ना एका machine वर भांडू देत नाही?

Section 4: Software बनतं कसं? Bugs असतातच का? बदलांची नोंद कशी राहते? आणि engineer चा estimate नेहमी चुकीचा का असतो?

शेवटचा chapter तुमच्यासाठी सगळ्यात महाग आहे: जेव्हा तुम्ही कोणाकडून software बनवून घ्याल, तेव्हा हा part सांगेल की काय normal आहे, काय हलगर्जीपणा, आणि पैसा कशाचा द्यायचा.

SECTION 3: MACHINE ला सांगणं की काय करायचं

सात chapters. माणसाच्या विचारापासून machine च्या numbers पर्यंतचा पूर्ण रस्ता: languages, भाषांतर करणारे, आणि operating system.

हे तुमच्या business decision मध्ये कुठे येईल: “कुठल्या language मध्ये बनवायचं,” “कुठला platform धरायचा,” “हे फुकट tool भरवशाचं आहे का?” हे सगळे founder चे निर्णय आहेत, engineer चे नाही, कारण यांतून hiring, खर्च आणि वेग ठरतो. या section नंतर तुम्ही हे निर्णय समजून घेऊ शकाल, ऐकून नाही.

Chapter 3.1 [SPINE]: भाषांतराची problem

Part 1 चं शेवटचं सत्य हे होतं: CPU ला फक्त numbers वाली पावलं समजतात. 3 म्हणजे जोडा, 7 म्हणजे उडी. बस. या numbers वाल्या भाषेचं नाव **machine code**.

मग 1940s च्या पहिल्या programmers नी काय केलं? खरंच numbers च लिहिले. कागदावर पावलं विचार करायचे, मग प्रत्येक पावलाचा number ठरवायचे, मग switches आणि cards ने machine मध्ये भरायचे. एका छोट्या recipe ला आठवडा लागायचा, आणि एक number चुकला तर सगळा कचरा; आणि चूक **शोधणं** लिहिण्यापेक्षाही अवघड.

आता तुम्ही 1950 चे engineer आहात. हा त्रास रोज सहन करताय. काय कराल?

पहिलं पाऊल स्वतःच दिसतं: numbers ना **नावं** द्या. “3” ऐवजी लिहा “ADD”, “7” ऐवजी “JUMP”. माणूस नाव वाचू शकतो, चूक दिसते. पण machine ला तर नावं समजत नाहीत... मग एक छोटा program लिहा जो नावाला परत number बनवेल. हे भाषांतर इतकं सरळ आहे (प्रत्येक नाव = एक number) की program सोपा होता. या नावांवाल्या भाषेचं नाव **assembly**.

थांबा. इथे जे झालं ते नीट बघा, कारण हा पूर्ण software दुनियेचा जन्म आहे: **भाषांतराचं काम स्वतःच एक recipe बनलं**. Machine आपल्याच भाषेचा पूल स्वतः बांधते.

आता पुढचा त्रास. नावं numbers पेक्षा बरी, पण विचार अजूनही machine च्या पातळीवर करावा लागतो: “हे उचला, तिथे ठेवा, जोडा, उडी मारा.” माणूस असा विचार करत नाही. माणूस विचार करतो: “प्रत्येक ग्राहकाचं bill जोडा आणि सगळ्यात मोठं दाखवा.” एक ओळ, जिच्यात machine ची शंभर पावलं लपली आहेत.

मग स्वप्न बनलं: अशी भाषा जी माणसाच्या विचाराच्या **जवळ** असेल, आणि एक जाडजूड भाषांतरकार जो त्या एका ओळीला machine च्या शंभर पावलांमध्ये उघडेल. हा भाषांतरकार आता सरळ नाव-बदल नाही; हे खरं भाषांतर आहे: समजणं, तोडणं, जुळवणं. अवघड program आहे. पण एकदा बनला (leverage आठवतंय?), की प्रत्येक programmer कायमचा माणसाच्या पातळीवर लिहू शकतो. अशा भाषांना **programming languages** म्हणतात (Python, Java, C: पुढच्या chapters मध्ये भेटतील).

हे बनलं (1957 पासून), आणि तेव्हापासून पूर्ण industry याच शिडीवर चढते आहे: machine पासून दूर, माणसाच्या जवळ. Post office ची शिडी आठवा: तुम्ही फक्त पत्ता लिहिता; postman, sorting केंद्र, गाडी, हवाई थैली, हे सगळं खाली आपापलं काम करतं. तुम्हाला त्यातलं काही दिसत नाही. आणि शिडीचं

शेवटचं पाऊल तुम्ही रोज बघता: तुम्ही Claude ला मराठीत सांगता आणि code बनतो. ते याच 70 वर्ष जुन्या शिडीचं सगळ्यात नवं पाऊल आहे.

इथे लोक काय चुकीचं समजतात

"Computer आता English समजायला लागला आहे." नाही. CPU आजही तेच numbers समजतो जे 1950 मध्ये समजायचा. बदललं ते भाषांतर: मधले मजले इतके चांगले झाले की माणसाला machine दिसणं बंद झालं. AI तुमच्या मराठीतून code बनवतो, तेव्हाही शेवटी तेच 3 आणि 7 वाले numbers चालतात. शिडी लांब झाली; जमीन तीच.

MAP वर

कोण कमावतं: प्रत्येक मजल्यावर वेगळे लोक. Machine code च्या जवळ जगातले सगळ्यात कमी लोक (chip companies मध्ये, प्रति माणूस सगळ्यात उंच पगार). वरच्या मजल्यांवर crores लोक (सामान्य software jobs). आणि आता सगळ्यात वरचा मजला (AI कडून code बनवून घेणं) सगळ्यात नवा आहे; तिथे गर्दी अजून कमी आहे आणि नियम बनत आहेत. नवं पाऊल उघडताच आधी चढणारे सगळ्यात जास्त कमावतात; प्रत्येक वेळेस हेच झालं आहे.

स्वतः बघा (5 मिनिटं)

Laptop वर कुठलीही website उघडा, रिकाम्या जागेवर right-click करा, "View Page Source" दाबा. जे दिसलं, ती त्या page ची recipe आहे (HTML). घाबरू नका, वाचायचं नाही. फक्त हे बघा की प्रत्येक सुंदर page च्या खाली हा text बसलेला आहे, आणि हा text ही शेवटी numbers बनूनच चालला. पडद्यामागे डोकावण्याची सवय लावून घ्या.

विचार करा

1. (derivation) Machine ला सरळ मराठी किंवा English का नाही शिकवत, मधल्या भाषा काढून? अडचण काय आहे?

उत्तर: माणसाच्या भाषेत अर्थ अर्धा बोललेला असतो, अर्धा समजलेला. “थोडं मीठ टाका” मध्ये किती? Machine ला प्रत्येक पाऊल **exact** लागतं, कारण तिच्याकडे “समजून घे ना” वाला mode नाही (Chapter 1.5: सगळं आधीच ठरलेलं). म्हणून मधली भाषा लागते जिच्यात अर्थाचं एकच रूप असतं. AI इथे पहिला खरा प्रयत्न आहे: तो तुमच्या अर्धवट बोलण्याचा अर्थ **अंदाजाने** काढतो, मग exact code बनवतो. पण अंदाज कधीकधी चुकतो; म्हणून AI च्या जमान्यातही शेवटची जबाबदारी तीच आहे: exact काय हवंय, हे **तुम्हाला** माहीत पाहिजे. म्हणूनच तुम्ही हे पुस्तक वाचताय.

Chapter 3.2 [SPINE]: जवळजवळ-English मध्ये लिहिणं

आता ती गोष्ट समोरून बघून घ्या जिला जग एवढं घाबरतं: code. हा घ्या एक पूर्ण, खरा program (Python भाषा):

```
price = 300

if price > 500:
    print("mahag aahe")
else:
    print("theek aahe")
```

वाचा. तुम्ही हे **जवळजवळ** English सारखं वाचू शकता: “price नावाच्या खणात 300 ठेवा. जर price 500 पेक्षा मोठा असेल, तर ‘mahag aahe’ दाखवा, नाहीतर ‘theek aahe’ दाखवा.” चालवलात तर screen वर येईल: theek aahe.

बस. हे coding आहे. ना maths चा डोंगर, ना जादू. कडक व्याकरणात लिहिलेली recipe. या लिहिलेल्या recipe चं नाव **code**, आणि कडक व्याकरणाचं नाव **syntax**.

आता एक गोष्ट जी coding बाहेरच्या लोकांना कोणी सांगत नाही: **प्रत्येक program, WhatsApp पासून ChatGPT पर्यंत, फक्त तीन गोष्टींनी बनला आहे:**

- 1. ठेवा:** नावाच्या खणात value ठेवणं (`price = 300`). खणाचं नाव **variable**. खण म्हणजे तेच RAM चं मेज (Chapter 2.5); नाव माणसाच्या सोयीसाठी.
- 2. विचारा:** अटीवर रस्ता निवडणं (`if ... else ...`). अटीचं नाव **condition**. हे तेच “जर” आहे जे gates पासून बनलं होतं (Chapter 1.5), फक्त आता माणसाच्या रूपात.
- 3. पुन्हा करा:** एक काम पुन्हा पुन्हा करणं (**loop**):

```
for grahak in grahak_yadi:
    bill = bill_kadha(grahak)
    print(bill)
```


“प्रत्येक ग्राहकासाठी: bill काढा, दाखवा.” यादीत तीन असोत की 30 million, recipe त्याच तीन ओळी. **हीच** ती जागा जिथे leverage code मध्ये घुसतो: माणूस 30 million वेळा करू शकत नाही, loop करून टाकतो.

ठेवा, विचारा, पुन्हा करा. ढाब्याची पूर्ण व्यवस्था हीच आहे: डब्यांमध्ये सामान (ठेवा), “order मध्ये तिखट कमी लिहिलंय का?” (विचारा), आणि tandoor मध्ये एकामागून एक रोट्या (पुन्हा करा). आता कुठलंही software बघा आणि स्वतःला विचारा: यात काय ठेवलं जातंय, काय विचारलं जातंय, काय पुन्हा-पुन्हा होतंय. Instagram feed: posts च्या यादीवर loop, प्रत्येक post वर “याला दाखवायची का?” वाला if, आणि बघितलेल्यांचा हिशोब variables मध्ये. पूर्ण app तीन गोष्टींचं जाळं.

आणि bank चं खरं उदाहरण: दर महिन्याच्या 1 तारखेला लाखो खात्यांवर interest पडतो. आत काय आहे? एक loop (“प्रत्येक खात्यासाठी”), एक condition (“जर savings account असेल”), आणि काही variables (rate, balance, दिवसांची मोजणी). जो clerk 1960 मध्ये महिना लावायचा (Chapter 1.3), तो आज 20 ओळींचा loop आहे जो रात्री चालतो.

इथे लोक काय चुकीचं समजतात

“Coding साठी maths genius लागतो.” तुम्ही आता एक program वाचला आणि एकाचा अंदाज केला, कुठल्याही maths शिवाय. सामान्य software मध्ये बेरीज-वजाबाकीपेक्षा जास्त maths नसतंच. खरी skill maths नाही, **साफ विचार** आहे: कामाला इतक्या छोट्या, exact पावलांत तोडणं की एक मूर्ख पण वीज-वेगवान नोकर (machine) ती करू शकेल. ज्या माणसाला business ची प्रक्रिया साफ विचार करता येते, त्याला coding ची विचारसरणी आधीच येते; फक्त व्याकरण येत नाही.

MAP वर

रुपयाचा रस्ता: जगात ~30 million लोक code लिहून कमावतात. पण आता खरी गोष्ट: AI नंतर **व्याकरण स्वस्त** होतंय (Claude ला syntax येतं) आणि **साफ विचार महाग** होतोय. ज्या माणसाला माहीत आहे **काय** बनवायचं आणि ते exact पावलांत तोडता येतं, तो आता व्याकरण न शिकताही बनवून घेऊ शकतो. तुमच्या या पुस्तकाची पूर्ण bet याच ओळीवर आहे.

स्वतः बघा (5 मिनिटं)

वरच्या पहिल्या program मध्ये 300 चं 800 करा (डोक्यात). आता काय छापेल? आता 500 चं 800 करा, price 800 च राहू द्या. आता काय? (लक्ष द्या: प्रश्न “मोठा आहे का” आहे, “बरोबर आहे का” नाही.) दोन्ही उत्तरं न अडता दिलीत (पहिलं: mahag aahe; दुसरं: theek aahe, कारण $800 > 800$ खोटं आहे), तर तुम्ही आज code **trace** केला; हीच debugging ची पहिली skill आहे.

विचार करा

1. (derivation) एक bracket किंवा comma सुटला तर पूर्ण program का थांबतो? माणूस तर अर्ध्या चुकीच्या spelling ची चिठ्ठीही वाचतो.

उत्तर: माणूस अर्थाचा अंदाज लावतो कारण त्याच्याकडे जगाची समज आहे. भाषांतरकाराकडे फक्त व्याकरणाचे नियम आहेत; bracket शिवाय त्याला कळतच नाही की “विचारा” कुठे संपलं आणि “करा” कुठे सुरू झालं. आणि अंदाज लावणं **मुद्दाम** ठेवलेलं नाही: bank च्या code मध्ये भाषांतरकाराचा चुकीचा अंदाज करोडोंचा पडू शकतो. बरं आहे तिथेच थांबवा, माणसाला विचारा. कडक व्याकरण शिक्षा नाही; ती exactness ची किंमत आहे, आणि exactness च ती गोष्ट आहे जिच्यामुळे machine वर विश्वास ठेवता येतो.

Chapter 3.3 [DEPTH]: भाषांतरकाराचे दोन प्रकार

(DEPTH chapter. वगळू शकता; परत आलात की “compiled vs interpreted” वाली प्रत्येक tech चर्चा उघडेल.)

एक पुस्तक मराठीत आहे, तुम्हाला ते English वाचकांपर्यंत पोहोचवायचं आहे. दोन रस्ते आहेत, दोन्ही तुम्ही स्वतः विचार करू शकता:

पहिला: पूर्ण पुस्तकाचं भाषांतर आधी करून घ्या. वेळ लागेल, पण त्यानंतर प्रत्येक English वाचक सरळ वाचेल, पूर्ण वेगाने. आणि भाषांतरकार पूर्ण पुस्तक बघतो, म्हणून चुका आधीच सापडतील, छापण्याआधी.

दुसरा: वाचकाशेजारी एक दुभाषी बसवा. सुरुवात लगेच होईल: एक ओळ ऐका, एक ओळ बोला. पण प्रत्येक वाचनाला भाषांतर पुन्हा होईल, म्हणून वेग कमी. आणि चूक तेव्हाच सापडेल जेव्हा ती ओळ येईल; पुस्तकाच्या मध्यात अडकू शकता.

Code चेही हेच दोन रस्ते आहेत:

Compiler = पूर्ण भाषांतर आधी. Code एकदा machine code मध्ये बदलला, आता ती file सरळ CPU वर धावते. वेगवान. Games, browsers, operating systems असे बनतात (भाषा: C, C++, Rust). तयार file चं नाव **binary** किंवा **executable**: .exe वाली file हीच आहे, भाषांतर झालेली recipe.

Interpreter = शेजारी बसलेला दुभाषी. लिहा आणि लगेच चालवा, बदला आणि पुन्हा चालवा. आराम आहे, पण चालताना प्रत्येक ओळीचं भाषांतर होतं, म्हणून हळू (10-100 पट पर्यंत). Python असं आहे.

मग सौदा साफ आहे: **compiler = मेहनत आधी, वेग नंतर. Interpreter = आराम आधी, वेगाची किंमत नंतर.** स्वयंपाकाची उपमा: लग्नाच्या पंगतीसाठी सगळी तयारी आदल्या दिवशी करून ठेवता (चिरून, वाटून, मोजून), मग वाढताना विजेचा वेग. घरच्या रोजच्या जेवणात चालता-चालता ठरवता, आज काय बनवायचं. दोन्ही बरोबर; प्रसंग वेगळे.

कुठला बरोबर? चुकीचा प्रश्न. **कुठल्या कामासाठी**, हा प्रश्न विचारा. Game ला वेग हवा: compiler. रोज बदलणारं छोटं काम: interpreter. (जगात मधले रस्तेही आहेत; Java आणि JavaScript अर्ध आधी, अर्ध चालता-चालता भाषांतर करतात. पण मूळ कल्पना याच दोन.)

Android phone वर कधी update नंतर लिहून येतं “Optimising apps 1 of 132...” ते हेच आहे: phone आपल्या CPU साठी apps चं भाषांतर आधीच करून ठेवतोय, म्हणजे नंतर प्रत्येक app वेगाने उघडेल. तुम्ही compiler ला काम करताना बघितलं आहे; फक्त नाव माहीत नव्हतं.

इथे लोक काय चुकीचं समजतात

”Python हळू आहे, मग AI सारखी जड कामं Python मध्ये का होतात?” चाल अशी आहे: Python फक्त **steering** आहे. AI चा खरा जड हिशोब (numbers चा गुणाकार, billions वेळा) compiled code मध्ये लिहिलेला आहे (C/C++ च्या libraries); Python फक्त त्याला बोलावतो. हळू भाषेतून वेगवान engine चालवलं जातंय. Abstraction पुन्हा: आराम वर, वेग खाली, दोन्ही एकत्र.

MAP वर

रुपयाचा रस्ता: server वर प्रत्येक second च्या CPU चं भाडं लागतं (Part 5 मध्ये पूर्ण हिशोब). हळू code = जास्त seconds = मोठं bill. म्हणून companies मोठ्या झाल्या की आपले सगळ्यात जास्त चालणारे भाग compiled भाषांमध्ये **पुन्हा** लिहितात, फक्त bill घटवण्यासाठी. Founder साठी धडा: सुरुवातीला आरामवाली भाषा (वेगाआधी जिवंत राहणं गरजेचं), scale वर वेगवाली. चुकीच्या वेळी चुकीची निवड, दोन्ही बाजूंनी महाग.

स्वतः बघा (5 मिनिटं)

Laptop वर एक .exe file शोधा (कुठल्याही app च्या folder मध्ये) आणि एक .txt file. .txt उघडा: माणसाला वाचता येतं. .exe ला Notepad मध्ये उघडायचा प्रयत्न करा: विचित्र अक्षरांचा कचरा दिसेल, कारण ते machine code आहे. दोन दुनियांचा फरक स्वतःच्या डोळ्यांनी: एक माणसासाठी लिहिलेली, एक CPU साठी.

विचार करा

1. (derivation) Compiler चुका आधी पकडतो, interpreter चालताना. Bank चं core system कशाने बनावं, आणि दर आठवड्याला बदलणारी नवी कल्पना कशाने? कारणही द्या.

उत्तर: Bank: compiler वाली भाषा. तिथे चूक चालल्या **नंतर** सापडणं करोडोंचं पडू शकतं; आधी पकडणं हाच बचाव, आणि वेगही हवा. नवी कल्पना: interpreter वाली. तिथे सगळ्यात मोठा धोका हळू code नाही, **हळू शिकणं** आहे: दर आठवड्याला बदल करायचाय, लगेच चालवून बघायचाय. सौदा नेहमी तोच: चुकीची किंमत vs बदलाचा वेग. ही एक ओळ कुठल्याही tech निर्णयावर लावा; अर्ध उत्तर मिळेल.

Chapter 3.4 [SPINE]: इतक्या साऱ्या languages का आहेत

Python, Java, C, JavaScript, Swift, Kotlin, Go, Rust... शंभरपेक्षा जास्त आहेत. नव्या माणसाला वाटतं हा वेडेपणा आहे, किंवा fashion. नाही. कारण तेच आहे जे गाड्यांचं आहे.

Truck, bike, car, tractor: सगळे “जाण्याचं साधन” आहेत, तरी चार का? कारण प्रत्येक design एक **सौदा** आहे: truck सामान जास्त, वेग कमी; bike स्वस्त, पावसात भिजाल. एक साधन सगळं best करू शकत नाही, कारण खुब्या एकमेकींशी भांडतात.

Languages चे सौदेही मोजके आहेत, चार मुख्य:

वेग vs आराम. Machine च्या जवळ लिहा (C) तर वेगवान पण प्रत्येक गोष्ट स्वतः सांभाळा; दूर लिहा (Python) तर आराम पण हळू. (मागच्या chapter चा सौदा.)

स्वातंत्र्य vs सुरक्षा. काही भाषा तुम्हाला काहीही करू देतात (C: memory पर्यंत थेट हात); ताकद मिळते, पण चुकीचं दारही उघडं. काही बंधनं घालतात (Rust, Java); चुकांची पूर्ण जातच संपते.

सर्व कामं vs खास काम. काही प्रत्येक कामासाठी (Python, Java). काही एकाच कामासाठी बनल्या, पण त्या कामात राणी: SQL फक्त data ला प्रश्न विचारण्यासाठी (Part 4 मध्ये भेटेल), HTML फक्त page चं रूप सांगण्यासाठी.

जागेचा कब्जा. काही भाषा यासाठी चालतात कारण त्यांचा एका जागेवर कब्जा आहे: browser च्या आत फक्त JavaScript चालते, iPhone app साठी Apple ने Swift ठेवली आहे, Android साठी Kotlin. तिथे काम करायचं तर तीच भाषा.

बस. प्रत्येक language या चार सौद्यांची एक जोडणी आहे. “सगळ्यात चांगली language कुठली” हा प्रश्न “सगळ्यात चांगली गाडी कुठली” सारखाच आहे: काम सांगितल्याशिवाय उत्तर नाहीच.

एका project मध्ये वापरलेल्या सगळ्या भाषा आणि tools च्या जोडणीला **tech stack** म्हणतात (Chapter 0.3 वाला शब्द परत, आता छोट्या आकारात). Zomato च्या आत एकाच वेळेस किमान पाच भाषा जिवंत आहेत: app Kotlin/Swift मध्ये (phones चा कब्जा), मागचे servers Java/Go मध्ये (वेग + भरवसा), data चे प्रश्न SQL मध्ये, AI चं काम Python मध्ये, website JavaScript मध्ये. वेडेपणा नाही: प्रत्येक जागी तीच भाषा जी त्या जागेचा सौदा जिंकते. मंडईसारखं आहे: भाजीवाला हातगाडी वापरतो, घाऊक व्यापारी tempo, आणि बाहेरचा माल train ने येतो; कोणी “एकच साधन” वापरत नाही.

इथे लोक काय चुकीचं समजतात

लोक language ला धर्म बनवतात (“मी Python वाला”). Engineers चे हे वाद खरे असतात, पण तुम्ही founder च्या नजरेने बघा: language ही एक **hiring** decision आहे. Python/JavaScript वाल्यांची गर्दी मोठी (स्वस्त, लवकर मिळतो), Rust वाल्यांची छोटी (महाग, शोधावा लागेल). नव्या startup चा अर्धा निर्णय इथेच होतो, technology च्या आधी.

MAP वर

Scarcity चा उलटा खेळ बघा: COBOL 1959 ची भाषा आहे, कोणी शिकत नाही. पण जगातल्या banks चे जुने core systems तिच्यावरच उभे आहेत, आणि ते बदलणं इतकं धोक्याचं आहे की कोणी बदलत नाही. निकाल: उरलेल्या म्हाताऱ्या COBOL वाल्यांना banks जाड fees देतात. धडा: पैसा नव्यात पण आहे, आणि त्या जुन्यातही ज्याला सगळे सोडून गेले पण जग अजून ज्याच्यावर उभं आहे. Scarcity दोन्ही टोकांना बनते.

स्वतः बघा (5 मिनिटं)

कुठल्याही job site वर (LinkedIn/Naukri) search करा: “Python developer” आणि मग “COBOL developer”. मोजणी बघा, आणि जिथे salary दिसली तिथे तीही. आता जे दिसलं ते Chapter 0.2 च्या सूत्रात ठेवा: मागणी किती, पुरवठा किती. Language चा बाजार तुम्ही आत्ता स्वतः मोजला.

विचार करा

1. (derivation) Browser मध्ये फक्त JavaScript चालते; हा “जागेचा कब्जा” आहे. या कब्जाचा JavaScript वाल्यांच्या कमाईवर आणि गर्दीवर काय परिणाम झाला असेल? सूत्राने विचार करा.

उत्तर: कब्जाने मागणी पक्की केली: प्रत्येक website बनवणाऱ्याला JavaScript लागतेच, म्हणजे कामाचा size खूप मोठा. पण याच कारणाने **crores** लोकांनी ती शिकली, म्हणजे scarcity कोसळली. निकाल: काम सगळ्यात जास्त, सरासरी पगार मधला. उलट, ज्या skill ची मागणी मोठी **आणि** शिकणारे कमी (आज: AI systems नीट चालवणं आणि जोडणं), तिथे दोन्ही काटे वर. गर्दी जिथे पोहोचली तिथला पैसा average झालेला असतो; सूत्र नेहमी पुढची रिकामी जागा शोधायला सांगतं.

Chapter 3.5 [SPINE]: तो manager जो machine वाटतो

आता तुमच्या phone वर काय-काय चालू आहे? WhatsApp, गाणं, background मध्ये email check, screen चा touch, network ची ऐकीव. पण Part 1 मधून तुम्हाला माहीत आहे: CPU एका वेळेस **एका** recipe चं **एक** पाऊल चालवतो. मग हे सगळं **एकाच वेळेस** कसं?

आणि एक खोल अडचणही आहे. Chapter 2.4 च्या प्रश्नात दिसली होती: एका recipe ने दुसऱ्या recipe च्या memory त लिहिलं तर? शंभर recipes, एक मेज (RAM), कुठलीही रोकटोक नाही: एक खराब app पूर्ण phone पाडेल.

मग स्वतः design करा. तुमच्याकडे एक वेगवान CPU आहे आणि शंभर recipes आहेत ज्यांना चालायचं आहे. काय कराल? एका ढाब्याचा विचार करा ज्यात एकच चूल आहे आणि दहा cooks:

पाळ्या वाटा. प्रत्येक recipe ला थोडा वेळ द्या: 10 milliseconds WhatsApp, 10 गाण्याला, 10 screen ला, मग परत. CPU इतका वेगवान आहे (billion पावलं/second) की प्रत्येक recipe ला वाटतं machine फक्त तिचीच आहे. “एकाच वेळेस” हा भास आहे; खरं म्हणजे वीज-वेगाची पाळी-पाळी. (या पाळ्या वाटण्याचं नाव **scheduling**.)

भिंती बांधा. प्रत्येक recipe ला मेजाचा आपला हिस्सा द्या, आणि नियम: आपल्या हिश्याबाहेर हात घातला तर तिथेच मारून टाका (हेच “app crash” आहे). एक app मेलं, बाकीची वाचली.

सामायिक गोष्टींचा दरबान व्हा. Screen एक आहे, speaker एक, network एक, storage एक. Recipes ना थेट हात लावू देणार नाही; त्या विनंती करतील, दरबान करेल. Files ची व्यवस्थाही हाच दरबान ठेवतो (**file system**), आणि प्रत्येक hardware शी बोलणारा त्याचा तुकडा म्हणजे **driver**.

आता बघा तुम्ही काय बनवलंत: एक **मालक** recipe, जी बाकी सगळ्या recipes च्या वर बसते, वेळ वाटते, भिंती ठेवते, दरबानी करते. हेच **operating system (OS)**. Windows, Android, iOS, Linux: हीच चार नावं जगातल्या जवळजवळ प्रत्येक machine चे मालक आहेत.

आणि आता ती गोष्ट जी याची खरी ताकद आहे: app लिहिणारा screen, touch, network, file, **काहीही** थेट शिवत नाही. तो फक्त OS कडे मागतो. म्हणजे app **OS साठी** लिहिली जाते, machine साठी नाही. म्हणूनच iPhone ची app Android वर चालत नाही: machine जवळजवळ सारखीच आहे, **मालक** वेगळा आहे, आणि app मालकाच्या भाषेत लिहिली होती.

इथे लोक काय चुकीचं समजतात

"Android/iOS म्हणजे phone चा brand-type." नाही, तो मालक आहे, आणि मालक असणं हे जगातलं सगळ्यात मोठं business-पद आहे. जो OS चा मालक, तो ठरवतो की त्या machine वर कुठलं app येईल, कसं येईल, आणि (खरी गोष्ट) पैशाचा किती हिस्सा रस्त्यात कापला जाईल. Apple प्रत्येक app च्या कमाईतले 15-30% घेतो, फक्त दरवाजाचा मालक म्हणून. या पदाचं नाव **platform**, आणि त्या कापलेल्या हिश्याचं टोपणनाव platform tax. जकात-नाक्यासारखं: रस्ता ज्याचा, जकात त्याची.

MAP वर

साखळी बघा: तुम्ही 100 रुपयांची in-app purchase करता (Level 1 पासून पैसा घुसला). App वाल्याला ~70-85 मिळाले, Apple/Google ने 15-30 रस्त्यात घेतले. App वाला आपल्या cloud आणि tools वाल्यांना देतो. OS चा मालक प्रत्येक व्यवहारात बसलेला आहे, ते app न बनवता. म्हणून companies OS/platform ची लढाई मरेपर्यंत लढतात: Microsoft (Windows), Google (Android फुकट वाटला, मालक होण्यासाठी), Apple. जो दरवाजा जिंकला, तो जकात वसूल करतो.

स्वतः बघा (5 मिनिटं)

Phone च्या Settings मध्ये Battery उघडा: कुठल्या app ने किती खाल्लं. मग Apps मध्ये जाऊन एखाद्या app च्या "Permissions" बघा: camera, location, contacts. हे दोन्ही पाने OS च्या manager-वह्या आहेत: कोणाला किती पाळी मिळाली, कोणाला कुठली किल्ली. मालकाचा हिशोब तुम्ही स्वतः वाचू शकता.

विचार करा

1. (derivation) Google, Android बनवायला billions खर्च करून, तो फुकट का वाटतो? (Chapter 0.1 ची विचारसरणी लावा: पैसा कुठे घुसतो, कुठे वाहतो.)

उत्तर: कारण Android विकून कमावणं छोटा खेळ होता; **मालक** होणं मोठा. फुकट OS मुळे जगातले 70% phones Google चे दरवाजे बनले: त्यांची search, त्यांचं Play Store (15-30% जकात), त्यांच्या ads, त्यांचा data. एकदा बनवा, billions phones वर चाले, प्रत्येक phone कमाईचा रस्ता: leverage चा पूर्ण काटा. Founder साठी धडा: कधीकधी product फुकट देणं हाच सगळ्यात मोठा business असतो, **जर** फुकट गोष्ट तुम्हाला दरवाजाचा मालक बनवत असेल. (फुकटचा पूर्ण खेळ Chapter 3.7 मध्ये.)

Chapter 3.6 [DEPTH]: “चालू आहे” चा अर्थ काय

(DEPTH chapter. छोटा आहे. यानंतर “app बंद करा, restart करा” वाल्या प्रत्येक गोष्टीचा खरा अर्थ दिसू लागेल.)

एक प्रश्न जो सरळ वाटतो पण नाही: WhatsApp तुमच्या phone मध्ये “आहे” याचा अर्थ काय? दोन अगदी वेगळ्या गोष्टी आहेत:

Recipe ची file. Storage मध्ये (पक्कं गोदाम) पडलेली, ~100 MB. झोपलेली. वीज गेली तरी काही बिघडत नाही. ही **app** आहे.

चालती copy. तुम्ही icon दाबलात तेव्हा OS ने recipe ला RAM च्या मेजावर उतरवलं, तिचा program counter set केला, तिला पाळी द्यायला सुरुवात केली. आता ती **जिवंत** आहे: तिची स्वतःची memory, तिची आत्ताची अवस्था (कुठली chat उघडी, कुठपर्यंत scroll केलं). हिचं नाव **process**, आणि त्या आत्ताच्या पूर्ण अवस्थेचं नाव **state**. (हा शब्द मोठा आहे; database आणि AI पर्यंत पुन्हा पुन्हा येईल.)

नाटकाची उपमा: script ची वही कपाटात पडलेली असते (file). प्रयोग रंगमंचावर चालू असतो (process). एकाच script चे दोन प्रयोग दोन गावांत एकाच वेळेस चालू शकतात, वेगवेगळ्या प्रेक्षकांसह, वेगवेगळ्या जागी थांबलेले. Script सामायिक; जिवंतपण वेगळं. Laptop वर दोन browser windows = एक program, दोन processes, प्रत्येकीची आपली state. दोन tabs मध्ये दोन वेगळे Gmail खाते उघडतात ते म्हणूनच.

आता रोजची भाषा या नव्या चष्म्याने वाचा:

“App उघडणं” = process जन्मणं (गोदामातून मेजावर). **“App not responding”** = process जिवंत आहे पण कुठल्यातरी कामात अडकली आहे; पाळी मिळते आहे पण उत्तर देत नाहीये. **“Force stop”** = OS process ला मारून टाकतो. File सुखरूप. **“Restart करा, बरं होईल”** = जुनी गोंधळलेली state वाली process मेली, नवी स्वच्छ state ने जन्मली. म्हणून restart एवढे आजार बरे करतो: आजार **state** मध्ये होता, file मध्ये नाही. **“Background apps”** = process जिवंत आहे पण screen वर नाही; OS तिला कमी पाळी देतो, किंवा झोपवून टाकतो.

इथे लोक काय चुकीचं समजतात

”App बंद केलं म्हणजे phone मधून गेलं.” आणि उलटही: “app delete केलं म्हणजे त्याचं सगळं गेलं.” दोन्ही चूक, कारण लोक **file** आणि **process** एक समजतात. बंद करणं process चा शेवट; file राहिली. Delete करणं file चा शेवट; पण app चा data (chats, settings) बहुतेक वेगळ्या जागी असतो, आणि cloud वरही. दोन नावं वेगळी ठेवा, अर्धा गोंधळ संपतो.

MAP वर

रुपयाचा रस्ता: cloud मध्ये तुम्ही **process च्या वेळाचं** भाडं देता. “Serverless” नावाच्या गोष्टीत हिशोबच हा आहे: तुमचा code जितके milliseconds process बनून चालला, तितका पैसा. Process = meter चालू, process संपली = meter बंद. Taxi सारखं: गाडी उभी आहे तर भाडं नाही, चालली तर meter फिरतं. Part 5 मध्ये cloud चं bill वाचाल, तेव्हा प्रत्येक ओळीमागे हाच प्रश्न असेल: किती processes, किती वेळ, केवढ्या मेजासह (RAM).

स्वतः बघा (5 मिनिटं)

Phone वर recent apps चं button दाबा (किंवा laptop वर Task Manager: Ctrl+Shift+Esc). ही जिवंत processes ची यादी आहे. एखाद्या app ला swipe करून मारा, मग पुन्हा उघडा: थोडं हळू उघडलं ना, आणि मागच्या जागेवर नाही? तुम्ही एका process चा पूर्ण जन्म-मृत्यू स्वतः घडवलात: state गेली, file मधून नवा जन्म झाला.

विचार करा

1. (derivation) Game मध्येच बंद पडला (process मेली). परत उघडला तर score जिथल्या तिथे मिळाला. State तर मेली होती; score कसा वाचला? Part 1 च्या कुठल्या chapter ची व्यवस्था आहे ही?

उत्तर: Game ने मरण्याआधी score storage मध्ये लिहिला होता (save), म्हणजे वेगवान-विसराळू RAM मधून हळू-पक्क्या गोदामात (Chapter 2.5). Process ची state नेहमी मरणारी आहे; जे वाचवायचं ते पक्क्या जागी लिहिणं हाच एकमेव रस्ता. याच एका कामाचं मोठं रूप म्हणजे **database**, जो Part 4 चा मुख्य नायक आहे: state ला अशा जागी ठेवणं जिथे process मेल्याने, machine जळाल्याने, काहीही बिघडत नाही.

Chapter 3.7 [SPINE]: फुकट गोष्टी exist का करतात

एक विचित्र यादी बघा: Android फुकट. WhatsApp फुकट. Google search फुकट. Python फुकट. Linux फुकट (आणि जगातले जवळजवळ सगळे servers त्याच्यावर चालतात). हे सगळं बनवायला billions लागले. Business च्या जगात कोणी billions लावून गोष्ट फुकट का वाटेल?

Chapter 1.6 मधून किल्ली उचला: software ची **copy** फुकट असते. मग फुकट वाटणं **शक्य** आहे. पण शक्य असणं कारण नाही. कारण चार आहेत, आणि चारही मध्ये पैसा कुठून तरी दुसरीकडून येत असतो. प्रत्येक फुकट गोष्टीवर हे चार प्रश्न चालवा:

- 1. फुकट गोष्ट कुठल्या दरवाजाची किल्ली आहे का?** Android फुकट, कारण प्रत्येक Android phone Google च्या ads आणि Play Store चा दरवाजा आहे (मागचा chapter). मंडईबाहेर फुकट चव देणारा तेच करतो: चव फुकट, पोतं विकत. या चालीचं business-नाव **loss leader**.
- 2. फुकट अर्ध आहे, पूर्ण विकलं जातं का?** फुकट app, पैशाने extra features. फुकट 15 GB, पैशाने 100. नाव: **freemium**. लाखो फुकट users पैकी 2-5% नी विकत घेतलं तरी business चालतो, कारण (पुन्हा तीच गोष्ट) फुकटवाल्यांना देण्याचा खर्च जवळजवळ शून्य.
- 3. तुम्हीच माल आहात का?** Facebook/Instagram/YouTube फुकट आहेत कारण खरा ग्राहक advertiser आहे, आणि विकली जाणारी गोष्ट तुमचं **लक्ष** आहे. जिथे गोष्ट फुकट आहे आणि विकायला काही दिसत नाही, तिथे बहुतेक तुम्हीच माल असता.
- 4. सगळे मिळून हत्यार बनवतायत का?** हा सगळ्यात वेगळा आहे: **open source**. Linux, Python, हजारो tools: यांचा code उघडा आहे; कोणीही वाचा, सुधारा, फुकट वापरा. का? कारण हत्यार सगळ्यांना लागतं आणि कोणाचा business नाही. Companies आपल्या engineers ना हे सुधारण्याचे **पगार** देतात, कारण त्यांचा पूर्ण business या हत्यारांवर उभा आहे आणि एकट्याने बनवणं महाग पडतं. गावाचा रस्ता सगळे मिळून बांधतात, दुकानं आपापली; रस्ता फुकट म्हणून कोणी गरीब होत नाही, रस्त्यामुळेच सगळी दुकानं चालतात.

खऱ्या जगातली सगळ्यात मोठी case: **Jio** ने 2016 मध्ये data जवळजवळ फुकट केला. तोटा? नाही; चाल क्रमांक 1: आधी पूर्ण देश दरवाजाच्या आत, मग हळूहळू भाव, आणि वर apps/platforms चा महाल. तोच खेळ Google ने Android शी खेळला, Amazon delivery च्या तोट्याने खेळतो. फुकट हे हत्यार आहे, आणि ते तोच चालवू शकतो ज्याच्याकडे लांब श्वास (पैसा) आणि पुढचा नकाशा आहे.

इथे लोक काय चुकीचं समजतात

दोन उलट्या चुका. पहिली: “फुकट आहे म्हणजे भिकार असेल.” Linux हे जगातलं सगळ्यात तपासलेलं software आहे; तुमची bank, तुमचा phone (Android आत Linux च आहे), सगळं त्याच्यावर. दुसरी: “फुकट आहे म्हणजे फुकट आहे.” नाही; किंमत दुसरीकडे आहे: तुमचं लक्ष, तुमचा data, किंवा दरवाजाची जकात. फुकट दिसलं की नेहमी विचारा: या साखळीत पैसा कुठे घुसतोय? (Chapter 0.3 चा प्रश्न; सगळीकडे कामी येतो.)

MAP वर

तुमच्यासाठी याचा सरळ अर्थ: तुमच्या startup चं पूर्ण हत्यार-घर **फुकट** आहे. Linux, Python, VS Code, Git: billions dollars चं tool-stack, किंमत शून्य. पन्नास वर्षांपूर्वी business काढायला factory लागायची; आज Level 3 चा पूर्ण कारखाना फुकट मिळतो, आणि Level 4 (AI) चं भाडं महिन्याला काही हजार. इतिहासात इतकी स्वस्त entry कधीच नव्हती. Scarcity आता हत्यारात नाही; ती **कल्पनेत** आणि **execution** मध्ये सरकली आहे. हत्यार सगळ्यांकडे आहे; चालवता सगळ्यांना येत नाही.

स्वतः बघा (5 मिनिटं)

तुमच्या phone ची कुठलीही 5 फुकट apps घ्या. प्रत्येकीवर चार-प्रश्न test चालवा: दरवाजा? freemium? तुम्ही-माल? open source? प्रत्येक app कुठल्या तरी एका (किंवा दोन) खणात बसेल. जी कुठल्याच खणात बसत नाही, तिच्यापासून सावध: एकतर तुम्हाला साखळी दिसत नाहीये, किंवा ती app असा data विकते आहे जो दिसत नाही.

विचार करा

1. (derivation) WhatsApp ने सुरुवातीला वर्षाला 1 dollar भाव ठेवला होता. मग Facebook ने 19 billion dollars ना विकत घेऊन तोही काढून टाकला. एवढे पैसे देऊन फुकट का केलं? साखळी लिहा.

उत्तर: Facebook चा माल लक्ष आणि data आहे, आणि त्याची सगळ्यात मोठी भीती होती की messaging चा दरवाजा दुसरा कोणी (Google, किंवा स्वतः WhatsApp मोठा होऊन) घेईल. 19 billion ही दरवाजाची किंमत होती; 1 dollar/वर्ष हा अडथळा होता जो प्रत्येक नव्या user ला थांबवायचा, म्हणून काढला: users जेवढे जास्त, जाळं तेवढं मोठं, दुसऱ्याचं घुसणं तेवढं अवघड. (याचं नाव network effect; Part 5 मध्ये पूर्ण भेटेल.) धडा: मोठी company एखाद्या फुकट गोष्टीवर billions उधळते, तेव्हा ती गोष्ट खर्च नाही, **किल्ला** आहे.

SECTION 4: SOFTWARE बनतं कसं

पाच chapters. आतापर्यंत बघितलं की code काय आहे. आता बघायचंय की खरं software **बनतं** कसं: चुका, बदल, आणि नेहमी-चुकणाऱ्या estimates च्या मधून.

हे तुमच्या business decision मध्ये कुठे येईल: एक दिवस तुम्ही कोणाकडून (माणूस किंवा AI कडून) software बनवून घ्याल. त्या दिवसासाठी हा section आहे: bug आल्यावर किती राग योग्य आहे, “झालं तर होतं” चा अर्थ काय, deadline दुप्पट का होते, आणि पैसा कशाचा द्यायचा: वचनाचा की चालत्या तुकड्याचा. जो founder हे जाणत नाही, तो एकतर लुटला जातो किंवा चुकीच्या माणसावर ओरडतो.

Chapter 4.1 [SPINE]: Software मध्ये bugs असतातच का

पूल पडत नाहीत (बहुतेक). घड्याळं चालत राहतात. मग software, जे जगातले सगळ्यात शिकलेले लोक बनवतात, रोज कुठे ना कुठे का तुटतं? हा हलगर्जीपणा आहे का?

नाही. ही **मोजणी** आहे. स्वतः बघा.

Chapter 1.3 आठवा: machine तेच करते जे **लिहिलं** आहे, ते नाही जो तुमचा **अर्थ** होता. मग bug ची व्याख्या एका ओळीत: **जे लिहिलं आणि जे हवं होतं, यांच्यामधली फट**. आता प्रश्न हा की एवढे हुशार लोक फट का सोडतात?

कारण recipe लिहिणाऱ्याला **प्रत्येक** परिस्थितीचं उत्तर आधीच लिहावं लागतं. आणि परिस्थिती मोजणीत bomb सारख्या फुटतात. एक छोटा form विचार करा: नाव, वय, pincode. नाव रिकामं असेल तर? 500 अक्षरांचं असेल? Emoji असेल? वय 0 असेल? Minus 5? 200? Pincode मध्ये अक्षरं असतील? User ने अर्धा भरला आणि network गेलं? दोनदा submit दाबलं? तीन fields चे दहा-दहा प्रकार घेतले तरी हजार जोड्या झाल्या. खऱ्या app मध्ये अशा **लाखो** जोड्या असतात, आणि लिहिणारा माणूस प्रत्येक जोडी डोक्यात खेळवू शकत नाही. जी जोडी त्याच्या डोक्यात कधी आलीच नाही, तीच एका दिवशी कुठल्या user च्या हाती लागते. त्या दिवशी “bug सापडला.” (अशा कोपऱ्यातल्या परिस्थितीचं नाव **edge case**, आणि program चं अचानक मरणं म्हणजे **crash**. Bug शोधून दुरुस्त करणं: **debugging**. नाव कुठून आलं: 1947 मध्ये एक खरा पतंग, moth, machine मध्ये अडकलेला सापडला होता; नाव चिकटलं.)

आता पुलाशी तुलना करा, फरक दिसेल. पुलाकडे परिस्थिती कमी आहेत: गाडी जड किंवा हलकी, वारा जोराचा किंवा हळू, सगळं एकाच physics च्या आत. Software कडे परिस्थितींचा स्फोट आहे, आणि प्रत्येक नवं feature जुन्या सगळ्यांसोबत नव्या जोड्या बनवतं. म्हणून 10 पट मोठं software 10 पट नाही, 100 पट जास्त जोड्यांचं असतं.

मग सत्य हे आहे: **मोठ्या software मध्ये bugs असणं पक्कं आहे; प्रश्न फक्त हा की किती लवकर सापडतात आणि किती नुकसान करण्याआधी पकडले जातात**. चांगली company ती नाही जिचे bugs शून्य आहेत (अशी company नाहीच); चांगली ती जिची पकडण्याची आणि दुरुस्तीची व्यवस्था वेगवान आहे. पुढचा chapter तीच व्यवस्था आहे.

खऱ्या जगात हे असं दिसतं: 31 December च्या रात्री किंवा cricket final च्या वेळेस UPI/payment apps का अडखळतात? कारण “एकाच वेळेस इतके लोक” हीच एक edge case आहे जी सामान्य दिवशी कधी येत नाही. किंवा calendar चे bugs: 29 February दर चार वर्षांनी येतो आणि दर चार

वर्षांनी कुठे ना कुठे कुठलं तरी system पडतं, कारण कुठल्या तरी लिहिणाऱ्याच्या डोक्यात ती जोडी नव्हती.

इथे लोक काय चुकीचं समजतात

”Bug आला म्हणजे engineer भिकार.” कधीकधी खरं, पण बहुतेक नाही. नवा founder पहिल्या bug वर ओरडतो; समजूतदार founder विचारतो: किती वेळात कळलं, किती users पर्यंत पोहोचला, पुन्हा होऊ नये यासाठी काय व्यवस्था बनली? Team चा दर्जा bugs च्या मोजणीने नाही, **उत्तराच्या** वेगाने मोजला जातो. (आणि जो विकणारा म्हणतो “आमच्या software मध्ये bugs नसतात,” तो खोटं बोलतोय किंवा त्याने मोठं software बनवलंच नाही.)

MAP वर

Bug चा स्वतःचा बाजार आहे. Companies आपलं system तोडणाऱ्यांना **बक्षीस** देतात: bug bounty. Google/Apple/Microsoft एका गंभीर bug चे लाखो-कोटी रुपयांपर्यंत देतात, कारण चोरीच्या आधी चौकीदाराला मिळणं स्वस्त पडतं. India तली मुलं bug bounty तून घरबसल्या dollars कमावतात: scarcity (तोडण्याची नजर सगळ्यांकडे नसते) x size (system billions लोकांचं) = बक्षीस मोठं. Level 3 च्या आत हा एक पूर्ण पेशा आहे ज्यात degree कोणी विचारत नाही, फक्त हुनर.

स्वतः बघा (5 मिनिटं)

कुठल्याही app च्या search box मध्ये टाका: काहीच नाही (रिकामा search), फक्त space, एक emoji, 200 अक्षरांचा कचरा. बघा काय होतं. बहुतेक सगळं ठीक असेल (कोणीतरी ही जोडी विचारली होती), कधी विचित्र उत्तर मिळेल (विचारली नव्हती; तुम्ही edge case पकडलात). तुम्ही आता तेच करताय जे tester पगार घेऊन करतो: त्या जोड्या शोधणं ज्या लिहिणाऱ्याच्या डोक्यात आल्या नाहीत.

विचार करा

1. (derivation) दोन features वाल्या app मध्ये features च्या आपसातल्या जोड्या मोजा: 1. आता 10 features वर किती जोड्या? आणि यातून काढा: “बस एक छोटंसं feature अजून टाका ना” ऐकल्यावर engineer चा चेहरा का पडतो?

उत्तर: 2 features = 1 जोडी. 10 features = प्रत्येकीची प्रत्येकीशी = 45. Features 5 पट वाढले, जोड्या 45 पट. प्रत्येक नव feature जुन्या **सगळ्यांसोबत** नव्या जोड्या आणतं, आणि प्रत्येक जोडी ही नवी जागा आहे जिथे bug लपू शकतो. म्हणून “छोटं feature” छोटं कधीच नसतं: त्याची खरी किंमत त्याच्या जोड्यांमध्ये आहे. Founder साठी सूत्र: feature ची किंमत = बनवायचा वेळ + कायमचं वाढलेलं जोड्यांचं ओझं. म्हणूनच उत्तम products features **टाकल्याने** नाही, feature न टाकण्याच्या हट्टाने बनतात.

Chapter 4.2 [DEPTH]: चूक आधी शोधणं

(DEPTH chapter. याच्याशिवायही गोष्ट पुढे जाईल, पण जो founder “testing” चा अर्थ जाणतो, तो कधी कच्चा माल विकत घेत नाही.)

मागच्या chapter चं सत्य मान्य करा: bugs येणारच. मग आता engineer च्या खुर्चीत बसा आणि विचार करा: user पर्यंत पोहोचण्याआधी ते कसे पकडायचे?

पहिलं उत्तर सगळ्यांचं तेच असतं: “चालवून बघा.” ठीक आहे. चालवलं, form भरला, काम झालं. पाठवायचं?

थांबा. तुम्ही **एक** रस्ता चालवलात. मागच्या chapter च्या हजार जोड्यांपैकी एक. आणि पुढच्या आठवड्यात code बदलेल (code रोज बदलतो); मग पुन्हा सगळं हाताने चालवणार? प्रत्येक बदलानंतर, प्रत्येक रस्ता, कायम? माणसाकडून हे होत नाही. आणि जिथे माणसाकडून होत नाही, तिथे काय येतं? (Part 1 चं उत्तर: recipe.)

मग चाल ही आहे: **तपासण्याचं कामही code बनवा**. अजून एक recipe लिहा जी पहिल्या recipe ला चालवते आणि उत्तरं तपासते:

```
tapasa: beriz(2, 2) == 4
tapasa: beriz(-5, 5) == 0
tapasa: bill_kadha(rikami_yadi) == 0
```

अशा शंभर-दोनशे तपासण्या लिहून ठेवा (डोक्यात येईल ती प्रत्येक edge case: रिकामं, शून्य, minus, खूप मोठं). आता प्रत्येक बदलानंतर machine **सगळ्या** तपासण्या seconds मध्ये चालवते. कालचा code आजही बरोबर आहे की कोणी तोडला, याचं उत्तर रोज, फुकट, लगेच. या छोट्या तपासण्यांचं नाव **unit tests**; प्रत्येक बदलावर आपोआप सगळ्या चालण्याच्या व्यवस्थेचं नाव **CI**; आणि नव्या बदलाने जुनी गोष्ट तुटण्याचं नाव **regression** (engineers च्या तोंडी रोज असतो हा शब्द).

हेच testing चं खरं रूप: एकदा लिहा, कायम तपासा. (Leverage, पुन्हा तेच. चांगल्या engineer चं प्रत्येक हत्यार शेवटी leverage चं निघतं.) Hotel च्या kitchen ची उपमा: चांगला ढाबेवाला प्रत्येक ताट पाठवण्याआधी चव घेतो; पण मोठं hotel दर तासाला ठरलेली यादी तपासतं: gas, फ्रिजचं तापमान, तेलाचा वास. यादी एकदा बनते, रोज चालते. तीच यादी software मध्ये code बनून बसते.

पण एक प्रामाणिक गोष्ट: tests लिहिणं वेळ खातं; आजचा वेग कमी करतं, उद्याचा वाचवतं. मग किती लिहायच्या? हा engineering चा नाही, **business** चा प्रश्न आहे: चुकीची किंमत काय? Bank चा पैसे मोजणारा code: चूक करोडोंची; tests वर कंजूपणा वेडेपणा. नव्या कल्पनेचं पहिलं रूप जे 50 लोक

बघतील: कमी tests, वेग जास्त, चालेल. जाणीवपूर्वक निवडलेला सौदा शहाणपण; न बघता सोडलेली testing हलगर्जीपणा. फरक तोच “जाणीवपूर्वक” आहे.

खऱ्या जगात: UPI किंवा bank चं नवं version सरळ तुमच्यापर्यंत येत नाही. आधी machine वर लाखो नकली व्यवहार चालतात (प्रत्येक edge case: शून्य रुपये, उलटी वेळ, दुहेरी नोंद), मग थोड्या खऱ्या users वर, मग सगळ्यांवर. म्हणून तुमचे पैशाचे apps इतके कमी पडतात: त्यांच्या मागे तपासण्यांची फौज रोज रात्री धावते. जो विश्वास तुम्हाला वाटतो, तो खरं म्हणजे tests ची मोजणी आहे.

इथे लोक काय चुकीचं समजतात

”Testing = launch च्या आधी एकदा सगळं check करणं.” नाही. ते फक्त एका रात्रीचं पहारा आहे. खरी testing **कायमची** आहे: code सोबत तपासण्यांची फौजही वाढत राहते, आणि प्रत्येक बदल त्यांच्यामधून जाऊनच बाहेर पडतो. म्हणून software बनवून घेताना विचारा: “tests किती आहेत, आणि प्रत्येक बदलावर आपोआप चालतात का?” या एका प्रश्नाने अर्थ कच्चे vendors गळून पडतात.

MAP वर

कोण कमावतं: QA/test engineers चा पूर्ण पेशा (India त लाखो लोक), आणि test-automation च्या tools चा आपला बाजार. पण नवं वळण बघा: AI आता tests **लिहिण्यात** सगळ्यात चांगला आहे, कारण edge cases विचार करणं pattern चं काम आहे. मग “हाताने test चालवणारा” पेशा आकसतोय, आणि “तपासण्यांची व्यवस्था design करणारा” महाग होतोय. तीच शिडी, पुन्हा: काम machine कडे, विचार माणसाकडे, पैसा विचारवाल्याकडे.

स्वतः बघा (5 मिनिटं)

Login form साठी 5 अशा तपासण्या कागदावर लिहा ज्या त्याला तोडू शकतील (रिकामा password? Emoji वाला email? 1000 अक्षरं? दोनदा submit? मध्येच net गेलं?). तुम्ही आता test plan लिहिलात, एका ओळीच्या code शिवाय. बनवून घेताना हीच यादी vendor ला द्याल, तर तो समजेल की समोर कच्चा ग्राहक बसलेला नाही.

विचार करा

1. (derivation) समजा tests लिहिणं पूर्ण फुकट झालं (AI लिहून देतो). मग bugs शून्य होतील का?

उत्तर: नाही. Tests फक्त ते तपासतात जे कोणाच्या तरी **डोक्यात** आलं. Bug तिथे असतो जिथे कोणाचंच डोकं गेलं नाही (Chapter 4.1 ची फट). फुकट tests ती फट छोटी करतील, संपवणार नाहीत: जी जोडी सुचलीच नाही, तिची तपासणीही लिहिली जाणार नाही; AI कडूनही तेव्हाच लिहिली जाईल जेव्हा त्याला विचारायचा रस्ता मिळेल. म्हणून zero-bug चं वचन physics नाही, marketing आहे. शक्य असलेलं best: फट छोटी करा, पकडण्याचा वेग वाढवा, नुकसानाची मर्यादा बांधा.

Chapter 4.3 [SPINE]: प्रत्येक बदलाची नोंद

तुम्ही कधी अशा files बघितल्या आहेत? `project_final.docx`, `project_final_v2.docx`, `project_FINAL_kharach.docx`. हसू येतं, पण हा एका खऱ्या problem चा कच्चा इलाज आहे: गोष्ट बदलत राहते, आणि आपल्याला जुनी रूपं लागतात.

Software च्या जगात ही problem राक्षस बनते. Code रोज बदलतो. दहा (किंवा दहा हजार) लोक **एकाच** code वर काम करतात. कालचं version चालत होतं, आजचं तुटलंय: **कोणी, काय, का** बदललं? आणि दोघांनी एकच file एकाच वेळेस बदलली तर कोणाचा बदल राहणार?

स्वतः व्यवस्था design करा. काय-काय लागेल?

1. प्रत्येक बदलाचा photo: काय बदललं, कोणी, कधी, आणि **का** (एका ओळीची नोंद).
2. कुठल्याही जुन्या photo वर परत जाण्याचं button.
3. एकत्र काम: मी माझ्या वेगळ्या copy वर नवं feature बनवीन, तू तुझ्यावर bug दुरुस्त कर, आणि नंतर दोघांची जोडणी होईल.

ही व्यवस्था बनलेली आहे; नाव **version control**, आणि हत्याराचं नाव **Git** (2005; त्याच माणसाने बनवलं ज्याने Linux बनवलं, स्वतःच्याच कामासाठी). त्याची भाषा तीन शब्दांत:

Commit = बदलाचा photo + नोंद. “Payment चा bug दुरुस्त केला” लिहून photo घेतला. इतिहासात कायमचा.

Branch = तुमची वेगळी copy जिथे तुम्ही कोणाला हात न लावता प्रयोग करता. मुख्य copy सुरक्षित राहते.

Merge = branch चं काम परत मुख्य copy त जोडणं. दोघांनी एकच ओळ बदलली असेल तर Git थांबवून विचारतो: कुठली ठेवू? (याचं नाव **merge conflict**.)

आणि **GitHub**? तो या सगळ्याचा अड्डा आहे: internet वर ठेवलेला सामायिक इतिहास, जिथे जगभरच्या teams (आणि open source चं पूर्ण जग) आपला code ठेवतात.

एक गोष्ट नीट समजून घ्या: हे फक्त “backup” नाही. ही **हिमतीची** व्यवस्था आहे. जेव्हा प्रत्येक बदल परत फिरवता येतो, तेव्हा प्रयोगाची भीती संपते. Git शिवाय engineer हात लावायला घाबरतो (“तोडून बसलो तर”); Git सोबत तो तोडतो, शिकतो, परत आणतो. वेगाचं खरं रहस्य safety net आहे. Bank ची जुनी रोजकीर्द (रोजमेळ) हीच शहाणपण होती: प्रत्येक नोंद कायम, खाडाखोड नाही; चूक झाली तर पुढची नोंद तिला उलटवते, पण इतिहास पुसला जात नाही. Git म्हणजे code ची रोजकीर्द.

सगळ्यात जवळचं उदाहरण तुम्ही स्वतः आहात: हे पुस्तक एका Git repo मध्ये बनतंय. प्रत्येक chapter चा प्रत्येक बदल commit आहे, नोंदीसह. जुनं Hinglish version पूर्णच्या पूर्ण archive मध्ये जिवंत आहे, एका command वर परत येऊ शकतं. जी व्यवस्था Microsoft च्या दहा हजार engineers ना सांभाळते, तीच तुमचं पुस्तक सांभाळते आहे, फुकट. (Chapter 3.7: हत्यार सगळ्यांसाठी सारखं फुकट आहे; फरक वापरण्यात आहे.)

इथे लोक काय चुकीचं समजतात

”हे engineers चं आतलं हत्यार आहे; founder ला काय.” चूक. GitHub हा कुठल्याही engineer किंवा team चा X-ray आहे. तिथे दिसतं: काम किती नियमित आहे (commits), नोंदी किती साफ आहेत, खरंच किती लोक काम करतायत. आणि तुम्ही AI कडून code बनवून घ्याल तेव्हा (आजची tools सरळ Git सोबत चालतात), commit इतिहास हाच तुमचा हिशोब आहे: काय बनवून घेतलं, कधी, कुठल्या नोंदीसह. वही वाचता आली पाहिजे, लिहिणारा कोणीही असो.

MAP वर

Company case: Microsoft ने 2018 मध्ये GitHub 7.5 billion dollars ना विकत घेतला. एका “code ठेवण्याच्या जागे”साठी एवढं का? कारण तिथे जगभरच्या engineers चं काम, सवयी आणि येणारे projects दिसतात: हा developers चा दरवाजा आहे (Chapter 3.5 च्या भाषेत: platform). आणि आता बघा: त्याच GitHub च्या वर Microsoft ने AI coding tools विकायला सुरुवात केली. आधी दरवाजा, मग दुकान. Pattern प्रत्येक वेळेस तोच.

स्वतः बघा (5 मिनिटं)

github.com उघडा (account शिवाय चालेल). Search मध्ये “linux” लिहा, पहिला result उघडा. तुम्ही जगातल्या सगळ्यात महत्वाच्या software चा खरा code बघताय. “Commits” वर click करा: लाखो बदलांचा इतिहास, प्रत्येकावर नाव, तारीख, नोंद. सगळ्यात नवा commit बहुधा कालचा असेल. हे 30 वर्षांपासून रोज बदलतंय आणि कधी तुटलं नाही: हीच version control ची ताकद.

विचार करा

1. (derivation) एकटा माणूस काम करतोय, team नाही. त्याला branch/merge ची काय गरज? (विचार करा: हिमतीची गोष्ट इथे कशी लागू होते.)

उत्तर: गरज तीच आहे, रूप छोटं. Branch ही त्याची प्रयोगशाळा आहे: “नवं design करून बघतो” वाली copy, जिच्यात fail होणं फुकट आहे, खरी गोष्ट सुरक्षित आहे. आणि इतिहास हा त्याचा दुसरा मेंदू: सहा महिन्यांनी “ही विचित्र ओळ का लिहिली होती” चं उत्तर commit च्या नोंदीत मिळतं. एकट्या माणसाकडे विचारायला दुसरं कोणी नाहीच; म्हणून त्याला नोंदीची गरज team पेक्षा **जास्त** आहे. (तुमचा second-brain repo पण हेच आहे: बदलत्या विचारांचं version control.)

Chapter 4.4 [DEPTH]: “माझ्या machine वर तर चालत होतं”

(DEPTH chapter. ही ओळ software जगातला सगळ्यात जुना विनोद आहे. हिच्यामागचं सत्य समजलं की “deploy” आणि “DevOps” सारखे जड शब्द हलके होतील.)

प्रसंग: engineer ने feature बनवलं, आपल्या laptop वर शंभर वेळा चालवलं, सगळं perfect. Server वर टाकलं: crash. User च्या phone वर: रिकामी screen. Engineer ची पहिली ओळ: “माझ्या machine वर तर चालत होतं...”

तो खोटं बोलत नाहीये. मग चूक कुठे आहे?

Chapter 3.5 मधून विचार करा. Recipe कधीच एकटी चालत नाही: ती OS कडे मागते, दुसऱ्या recipes ना (libraries) बोलावते, settings वाचते, files शोधते. म्हणजे recipe च्या भोवती एक पूर्ण **माहोल** आहे ज्याच्यावर ती टेकलेली आहे:

code स्वतः	(हा तर दोन्हीकडे सारखाच होता)
+ OS आणि त्याचं version	(laptop: Windows 11; server: Linux)
+ libraries चे versions	(laptop वर नवे, server वर जुने)
+ settings, passwords	(laptop वर test वाले, server वर खरे)
+ दुसऱ्या services	(laptop वर नकली bank; server वर खरी)

Code सारखा, माहोल वेगळा = निकाल वेगळा. “चालत होतं” चं पूर्ण सत्य: “**माझ्या** माहोलात चालत होतं.” या भोवतीच्या पूर्ण दुनियेचं नाव **environment**.

घरची डाळ आठवा. तुमच्या घरी दरवेळेस perfect बनते. तीच recipe मावशीच्या घरी बनवा: cooker वेगळा, आच वेगळी, मिठाचा डबा वेगळा. चव बदलली. Recipe खोटी बोलली नाही; **kitchen** बदललं होतं.

मग इलाज स्वतःच सुचतो: माहोलही code सारखा बनवा. लिहून ठेवा की या recipe ला काय-काय लागतं (OS, libraries चे exact versions, settings), आणि अशी व्यवस्था करा की प्रत्येक जागी, laptop पासून server पर्यंत, **तोच** माहोल बनेल. Recipe सोबत पूर्ण kitchen च pack करून द्या.

ही व्यवस्था बनली; डब्याचं नाव **container**, सगळ्यात प्रसिद्ध हत्यार **Docker**. एक डबा ज्यात code + त्याचा माहोल एकत्र बंद आहे; डबा जिथे उघडेल, आतली दुनिया सारखीच असेल. नाव ship च्या container वरूनच घेतलं: सामान कुठलंही असो, डबा एकसारखा, म्हणून प्रत्येक बंदराची crane तो

उचलते. McDonald's चं रहस्यही हेच: त्यांनी recipe नाही, **kitchen** standard केलं, म्हणून प्रत्येक शहरात चव तीच. आणि code ला laptop वरून खऱ्या server पर्यंत पोहोचवण्याच्या कामाचं नाव **deploy**; हे सगळं सांभाळणाऱ्या पेशाचं नाव **DevOps**.

इथे लोक काय चुकीचं समजतात

”Software = code ची file. File पाठवली, काम संपलं.” आता तुम्हाला माहीत आहे: software = code + माहोल. म्हणून “app बनली, आता live करणं म्हणजे बस एक button असेल ना” हा founder चा सगळ्यात महाग भ्रम आहे. Deploy हे स्वतःचं पूर्ण काम आहे: माहोल बनवणं, तपासणं, जुन्या version वरून नव्यावर विना-थांबा जाणं, चूक झाल्यास परत फिरणं. Estimate मध्ये याचा वेळ वेगळा विचारा, नाहीतर “झालं” आणि “चालतंय” मधले आठवडे तुम्हाला surprise करतील.

MAP वर

रुपयाचा रस्ता: हा त्रास इतका खरा आहे की त्याच्या इलाजाच्या companies billions च्या आहेत. Docker ने डबा दिला; मग प्रश्न आला “हजार डबे कोण सांभाळणार” तर Google ने Kubernetes उघडं केलं (open source: Chapter 3.7 ची चाल, सगळे त्याच्या cloud च्या पद्धतीवर यावेत म्हणून); आणि cloud companies चा (Part 5) अर्धा business हाच आहे: “माहोलाची कटकट आम्हाला द्या, भाडं द्या.” जिथे प्रत्येक team चं दुखणं एकसारखं असतं, तिथे platform बनतो, आणि platform जकात वसूल करतो. दुखणं शोधा; दुकान तिथेच उघडा.

स्वतः बघा (5 मिनिटं)

Play Store/App Store मध्ये कुठल्याही app चं page उघडा, खाली “Requires Android 12+” किंवा “Requires iOS 16+” शोधा. हे environment चं सगळ्यात सरळ रूप आहे: app म्हणते आहे “माझं kitchen किमान इतकं नवं हवं.” आता समजेल जुन्या phone वर नवी app का चालत नाही: code नाही, माहोल छोटा पडला.

विचार करा

1. (derivation) दोन मित्र, अगदी सारखा phone model, सारखं app version. एकाच्या phone वर app रोज crash, दुसऱ्याच्या कधीच नाही. Code सारखा, phone सारखा. मग काय वेगळं असू शकतं? किमान तीन गोष्टी मोजा.

उत्तर: माहोल अजूनही वेगळा आहे: (1) OS चं छोटं version/update वेगळं असू शकतं, (2) एकाचा storage जवळजवळ full आहे (लिहायला जागा नाही = विचित्र crashes), (3) permissions वेगळ्या (एकाने location नाकारलं; app चा तो रस्ता कधी test झालाच नव्हता), (4) भाषा/region setting वेगळं (तारखेचं रूप वेगळं = चूक), (5) background apps आणि battery-saver वेगळे. धडा: “सारखा phone” कधीच सारखा नसतो. म्हणून मोठे apps हजारो खऱ्या devices वर test होतात, आणि तरीही Chapter 4.1 चा नियम लागू राहतो: कुठली तरी जोडी सुटतेच.

Chapter 4.5 [SPINE]: Estimate नेहमी चुकीचा का असतो

जगातला प्रत्येक founder एकच गोष्ट सांगतो: “Engineer म्हणाला दोन आठवडे. दोन महिने झाले.” घर बांधणारेही हीच गोष्ट सांगतात, पण software मध्ये हे इतकं **पक्कं** का आहे? सगळे engineers खोटेवरडे आहेत का?

नाही. कारण software च्या स्वभावात आहे, आणि तुम्ही ते Part 1 च्या एका सत्यातून स्वतः काढू शकता.

Chapter 1.6: **copy फुकट आहे**. मग जे software आधीच बनलेलं आहे, ते पुन्हा कोणी बनवत नाही; copy करतात (किंवा विकत घेतात, किंवा open source उचलतात). याचा उलटा अर्थ नीट विचार करा: **software बनवण्याचं काम फक्त तेव्हा होतं जेव्हा ती गोष्ट आधी कधी बनलीच नाही**. प्रत्येक software project, व्याख्येनेच, काहीतरी **नवं** बनवणं आहे.

आता तुलना करा. Contractor शंभरावं घर बांधतोय: प्रत्येक भिंत आधीसारखी, surprises कमी, estimate बसतो. Engineer प्रत्येक वेळेस आपलं **पहिलं** “हे वालं” बनवतोय. आणि नव्या कामात surprises कुठे लपले आहेत, हे आधी दिसत नाही (दिसलं असतं तर surprise नसतं). Estimate चुकतो कारण तो त्या गोष्टीचं माप आहे जी अजून अंधारात आहे.

तीन गोष्टी अंधार अजून गडद करतात:

”90% झालं” चा भ्रम. दिसणारा भाग (screens, buttons) आधी बनतो आणि पटकन बनतो. न-दिसणारा (edge cases, deploy, दुसऱ्या systems शी जोड: Chapter 4.1 आणि 4.4 चे सगळे राक्षस) नंतर येतो. म्हणून “90% done” बहुतेक अर्ध्या रस्त्याचं नाव असतं: उरलेला 10% दिसतो छोटा, असतो मोठा.

बदलती मागणी. अर्थ बनल्यावर founder ते पहिल्यांदा चालवतो आणि त्याला समजतं की खरं तर काय हवं होतं. मागणी बदलली, माप जुनं; estimate चा काय दोष?

जोडणीचं दुखणं. दोन बनलेले तुकडे जोडणं “बस जोडून टाका” नसतं (Chapter 4.4: प्रत्येक तुकड्याचा आपला माहोल). जोडणी नेहमी मापापेक्षा जास्त खाते. लग्नाच्या स्वयंपाकासारखं: भात वेगळा perfect, रस्सा वेगळा perfect; पण दोनशे लोकांना एकाच वेळेस गरम वाढायचं म्हणजे वेगळंच युद्ध.

मग इलाज काय? Estimate सुधारणं नाही (तो कधीच पूर्ण सुधारणार नाही); **कामाची पद्धत** बदलणं:

छोटे तुकडे, चालता माल. सहा महिन्यांचं वचन घेऊ नका; दोन आठवड्यांचा असा तुकडा मागा जो **चालून** दाखवेल. चालता तुकडा खोटं बोलत नाही. प्रत्येक तुकड्यानंतर अंधार थोडा कमी, पुढचा estimate थोडा बरोबर. आणि सगळ्यात आधी तो तुकडा बनवा जो कल्पनेची सगळ्यात मोठी शंका test करतो, सगळ्यात सुंदर screen नाही. या पद्धतीचं नाव **agile** (विधी खूप आहेत; खरी गोष्ट हेच तुकडे), आणि पहिल्या छोट्या चालत्या रूपाचं नाव **MVP** (minimum viable product).

खऱ्या जगाची दोन्ही टोकं: सरकारी software चे मोठे प्रकल्प वर्षानुवर्षे उशीर आणि कधीकधी पूर्ण रद्द (billions बुडतात), कारण वचन मोठं आणि तुकडे नाहीत. उलट WhatsApp, Instagram, Zerodha: सगळ्यांनी आधी एक **छोटा** चालता तुकडा काढला (Instagram तर एका मोठ्या app मधलं कापलेलं एकच feature होतं: फक्त photo, filter, share). तुकडा चालला, मग वाढवला. मोठी वचनं बुडतात; छोटे तुकडे तरंगतात.

इथे लोक काय चुकीचं समजतात

”उशीर = team नालायक किंवा चोर.” आता तुम्हाला माहीत आहे: नव्या कामाचं माप हेच अंधाराचं माप. नालायकपणाच्या खऱ्या खुणा वेगळ्या आहेत: चालता तुकडा कधीच न दाखवणं, “सगळं एकदम शेवटी येईल” म्हणणं, दर आठवड्याला तेच “almost done.” आणि प्रामाणिक team ची खूण: छोटे तुकडे, प्रत्येक चालता, आणि वाईट बातमी **लवकर** सांगणं. आता तुम्ही दोन्ही ओळखू शकता.

MAP वर

हा chapter थेट तुमच्या खिशाशी जोडलेला आहे. Software बनवून घेताना (माणसाकडून किंवा AI कडून): (1) पैसा वचनावर नाही, चालत्या तुकड्यावर द्या; (2) पहिला तुकडा तो मागा जो सगळ्यात मोठी शंका test करतो; (3) “90% done” ऐकलं की demo मागा, screen नाही, **खरं** चालतं system; (4) estimate ला 2-3 ने गुणून आपला plan बनवा, आणि जी team स्वतःहून असं करून सांगते तिला मान द्या; ती प्रामाणिक आहे. या चार ओळी तुम्हाला त्या founders पासून वेगळं करतात जे लाखो घालवून शिकतात.

स्वतः बघा (5 मिनिटं)

तुमचा कुठलाही जुना अंदाज आठवा: लग्नाची तयारी, घर shift करणं, कुठलंही नवं काम. किती वाटलं होतं, किती लागलं? गुणोत्तर काढा (बहुतेक 1.5x-3x). आणि लक्ष द्या: जे काम **पहिल्यांदा** करत होतात, तेच सगळ्यात जास्त घसरलं. नव्या कामाचा अंधार फक्त software चा नाही, माणसाचा नियम आहे; software मध्ये फक्त **प्रत्येक** काम नवं असतं.

विचार करा

1. (derivation) AI आता code चे मोठे भाग minutes मध्ये लिहितो. मग आता estimates बरोबर होतील आणि software वेळेवर बनेल का?

उत्तर: थोडं बरं, पूर्ण नाही. AI ने **लिहिण्याचा** वेळ पाडला; पण estimate चा अंधार लिहिण्यात कमीच होता: तो **समजण्यात** आहे (काय हवंय), **जोडण्यात** (माहोल, दुसरी systems), आणि **तपासण्यात** (कुठली जोडी सुटली). उलट एक नवा धोका आला: AI मुळे “90% दिसणारं” आता तासांत बनतं, म्हणून भ्रम अजून वेगाने बनतो. Founder चा नियम तोच राहील: चालता तुकडा हाच पुरावा. हो, एक बदल खरा आहे: छोटा तुकडा आता **स्वस्त** बनतो, म्हणून प्रयोग जास्त करता येतात, चुकीचा रस्ता लवकर सोडता येतो. AI estimate बरोबर करत नाही; चुकीची किंमत पाडतो.

PART 2 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

भाषांतराची शिडी	machine code -> assembly -> languages -> AI; प्रत्येक मजला खालचा लपवतो
तीन गोष्टी	ठेवा (variable), विचारा (if), पुन्हा करा (loop): प्रत्येक program यांचंच जाळ
दोन भाषांतरकार	compiler (आधी, वेगवान) vs interpreter (सोबत, आरामाचा): चुकीची किंमत vs बदलाचा वेग
LANGUAGES	प्रत्येक भाषा चार सौद्यांची जोडणी; "कुठली best" नाही, "कुठल्या कामाला कुठली"
OS	machine चा मालक: पाळ्या, भिंती, दरबानी; मालक जकात वसूल करतो (platform tax)
PROCESS	file झोपलेली recipe; process जिवंत copy आपल्या state सह; restart = नवी स्वच्छ state
फुकटचा खेळ	दरवाजा / freemium / तुम्ही-माल / open source: पैसा नेहमी दुसरीकडून
BUG	लिहिलं vs हवं ची फट; जोड्यांचा स्फोट; मोजणी आहे, हलगर्जीपणा नाही
TESTING	तपासणंही code आहे; किती, हा business प्रश्न: चुकीची किंमत काय
VERSION CONTROL	प्रत्येक बदलाचा photo + परत जायचं button = हिमतीची व्यवस्था (Git, GitHub)
ENVIRONMENT	software = code + माहोल; म्हणून "माझ्या machine वर चालत होतं" (Docker, deploy)
ESTIMATE	प्रत्येक software पहिल्यांदा बनतंय; इलाज माप नाही, छोटे चालते तुकडे (MVP)

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, बोलून उत्तरं द्या:

1. iPhone ची app Android वर का चालत नाही?
2. Google Android फुकट का वाटतो?
3. "Bug-free software" चं वचन खोटं का आहे?
4. Software बनवून घेताना पैसा कशावर द्यायचा: वचनावर की कशावर? आणि का?

अडलात तर: 1 -> 3.5, 2 -> 3.5/3.7, 3 -> 4.1/4.2, 4 -> 4.5.

Part 3 मध्ये काय आहे

आतापर्यंतची पूर्ण गोष्ट **एका** machine च्या आत होती. पण तुमचा प्रत्येक message दुसऱ्या machine कडे जातो: शहरापलीकडे, समुद्रापलीकडे, अर्ध्या second मध्ये. कसा? Machines एकमेकींना शोधतात कशा? रस्त्यात कोण-कोण बसलं आहे? “Public रस्त्यावर private बोलणं” (तुमचा password!) कसं वाचतं? आणि internet चा मालक शेवटी कोण? Part 3: MACHINES चं जाळं.

PART 2 चा MINI-GLOSSARY

agile	छोट्या चालत्या तुकड्यांची पद्धत (4.5)
assembly	numbers च्या नावांची भाषा, machine जवळची (3.1)
binary/executable	भाषांतर झालेली, थेट चालणारी file (3.3)
branch / merge	प्रयोगाची copy / ती परत जोडणं (4.3)
bug	लिहिलं आणि हवं यांच्यामधली फट (4.1)
CI	प्रत्येक बदलावर सगळे tests आपोआप (4.2)
code	कडक व्याकरणात लिहिलेली recipe (3.2)
commit	बदलाचा photo + "का" ची नोंद (4.3)
compiler	पूर्ण भाषांतर आधी करणारा (3.3)
condition (if)	अटीवर रस्ता निवडणं (3.2)
container/Docker	code + माहोल एका डब्यात बंद (4.4)
crash	program चं अचानक मरणं (4.1)
debugging	bug शोधून दुरुस्त करणं (4.1)
deploy	code खऱ्या server पर्यंत पोहोचवणं (4.4)
DevOps	deploy आणि माहोलाची व्यवस्था-पेशा (4.4)
driver	hardware शी बोलणारा OS चा तुकडा (3.5)
edge case	कोपऱ्यातली दुर्मिळ परिस्थिती (4.1)
environment	recipe भोवतीची पूर्ण दुनिया (4.4)
file system	OS ची files ची व्यवस्था (3.5)
freemium	फुकट अर्ध, पूर्ण विकलं जातं (3.7)
Git / GitHub	version control चं हत्यार / त्याचा अड्डा (4.3)
interpreter	ओळी-ओळीने सोबत चालणारा दुभाषी (3.3)
loop	एक काम पुन्हा पुन्हा; code मधला leverage (3.2)
loss leader	फुकट चव, विक्री दुसरीकडे (3.7)
machine code	numbers ची भाषा जी CPU थेट समजतो (3.1)
merge conflict	दोन बदलांची टक्कर (4.3)
MVP	पहिलं छोटं चालतं रूप (4.5)
open source	उघडा code, सगळ्यांचं सामायिक हत्यार (3.7)
OS	machine चा मालक: पाळी, भित, दरबान (3.5)
platform (tax)	दरवाजाचा मालक आणि त्याची जकात (3.5)
process / state	जिवंत copy / तिची आत्ताची पूर्ण अवस्था (3.6)
regression	नव्या बदलाने जुनं काही तुटलं (4.2)
scheduling	CPU च्या पाळ्या वाटणं (3.5)
syntax (error)	कडक व्याकरण (त्याची चूक) (3.2)
tech stack	project च्या tools/भाषांची पूर्ण जोडणी (3.4)
unit test	छोटी आपोआप चालणारी तपासणी (4.2)
variable	नावाचा खण (3.2)
version control	बदलांच्या इतिहासाची व्यवस्था (4.3)